MODELO DE FALTAS

BookIT: sistema distribuído de reservas com recomendação inteligente

Projeto de Desenvolvimento de Software · 1.ª entrega · Grupo 6 · outubro de 2026

Nome completo	Número de estudante
Yassin Latif	20240626
Yosvany Costa Nunes	20230853
Francisco Silva	20241293
Lueji Covilhã	20241725

Arquitetura considerada

O modelo refere-se à arquitetura da Figura 1: um Nginx, duas ou mais instâncias da API de reservas, duas instâncias do serviço de IA e uma base de dados PostgreSQL com um primário e uma réplica, tudo em contentores Docker.

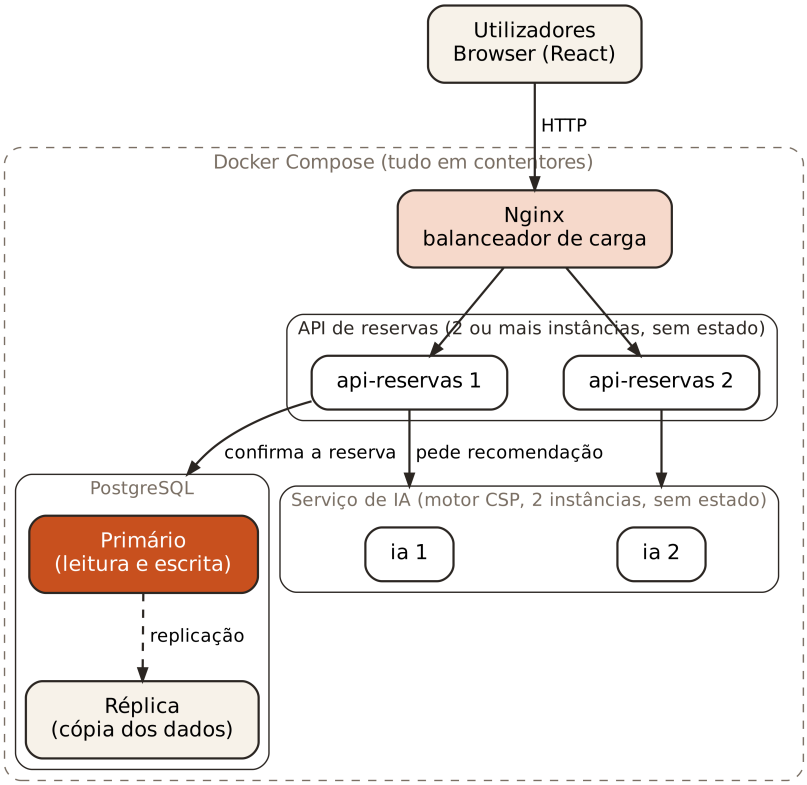


Figura 1. Arquitetura do sistema

Modelo de faltas

O sistema aguenta uma falha de cada vez numa das camadas e garante que nenhuma reserva confirmada se perde nem fica duplicada. Não promete disponibilidade total: o Nginx é único e a promoção da réplica é feita à mão. Estas limitações estão indicadas no fim.

Questão	Resposta
Que componentes podem falhar?	As instâncias da API de reservas, as instâncias do serviço de IA, o primário e a réplica da base de dados e a ligação entre componentes. O Nginx também pode falhar, mas não está protegido (limitação conhecida).
Que tipos de falha são considerados?	Paragem de um componente (processo ou contentor desligado) e perda temporária de ligação. Não consideramos comportamentos maliciosos nem dados corrompidos.
Quantas falhas em simultâneo?	Uma de cada vez. Com duas instâncias da API e duas da IA, perder uma de cada tipo não pára o serviço.
O que continua disponível após uma falha?	Pesquisar, reservar, cancelar e consultar reservas continuam a funcionar com uma só instância da API. Sem o serviço de IA, a recomendação passa a ser o primeiro horário livre. Se o primário da base de dados falhar, não se criam reservas até a réplica ser promovida.
Como é detetada uma falha?	Pelos health checks do Nginx às instâncias da API, por timeouts nas chamadas da API ao serviço de IA e à base de dados, e pelo estado da replicação no primário.
Como é feita a recuperação ou substituição?	Os contentores reiniciam e voltam ao conjunto (política de reinício do Docker). Se o primário falha, a réplica é promovida a primário à mão, a API passa a usar o novo primário e o nó antigo volta mais tarde como réplica.
Que informação pode ser perdida?	Os pedidos em curso no momento da falha podem falhar e o utilizador repete-os. As reservas confirmadas não se perdem, porque a réplica já tem a cópia (replicação síncrona).
Como se garante a consistência entre réplicas?	Só o primário aceita escritas e a replicação para a réplica é síncrona: a reserva só é confirmada depois de gravada nos dois nós. A base de dados recusa reservas sobrepostas (restrição de exclusão) e a chave de idempotência evita duplicados quando um pedido é repetido.

Cenários de falha a demonstrar

Para cada cenário indicamos os sete pontos pedidos no enunciado.

C1. Falha de uma instância da API

Componente que falha	Uma instância da API de reservas.
Como é detetada	O health check do Nginx deixa de ter resposta.
Que mecanismos mantêm o serviço	O Nginx envia os pedidos para as outras instâncias. As instâncias não guardam estado, por isso qualquer uma atende qualquer pedido.
Impacto nos utilizadores	Os pedidos em curso nessa instância falham e o utilizador repete. Os restantes não notam.
Perda de dados	Nenhuma.
Como é feita a recuperação	O contentor reinicia e volta ao conjunto.
Como se repõe a redundância	Fica reposta quando a instância volta a passar o health check.

C2. Falha de uma instância da IA

Componente que falha	Uma instância do serviço de IA.
Como é detetada	Timeout ou erro na chamada da API ao serviço de IA.
Que mecanismos mantêm o serviço	A API usa a outra instância. Se as duas falharem, sugere o primeiro horário livre.
Impacto nos utilizadores	A recomendação pode demorar mais ou ser mais simples. As reservas continuam a funcionar.
Perda de dados	Nenhuma.
Como é feita a recuperação	O contentor reinicia.
Como se repõe a redundância	A API volta a usar as duas instâncias.

C3. Falha do primário da base de dados

Componente que falha	O nó primário do PostgreSQL.
Como é detetada	A API não consegue ligar (timeout) e o health check do primário falha.
Que mecanismos mantêm o serviço	A réplica tem a cópia dos dados e é promovida a primário à mão. A API passa a usar o novo primário.
Impacto nos utilizadores	Não se criam reservas até à promoção. O pedido em curso falha e é repetido.
Perda de dados	Nenhuma nas reservas confirmadas (replicação síncrona). O pedido repetido não cria duplicado, por causa da chave de idempotência.
Como é feita a recuperação	Promoção da réplica e atualização da ligação da API.
Como se repõe a redundância	O nó antigo volta como nova réplica e ressincroniza.

C4. Falha da réplica

Componente que falha	O nó réplica do PostgreSQL.
Como é detetada	O estado da replicação no primário mostra que a réplica saiu.
Que mecanismos mantêm o serviço	O primário continua a funcionar. Como a replicação é síncrona, é preciso ajustá-la à mão para o primário voltar a confirmar escritas.
Impacto nos utilizadores	As escritas ficam em pausa até esse ajuste. As leituras continuam.
Perda de dados	Nenhuma.
Como é feita a recuperação	A réplica reinicia e volta a sincronizar com o primário.
Como se repõe a redundância	Repõe-se a replicação síncrona quando a réplica está atualizada.

C5. Corte de rede entre a API e a base de dados

Componente que falha	A ligação entre a API e o primário.
Como é detetada	Timeouts nas ligações.
Que mecanismos mantêm o serviço	A API devolve um erro controlado e tenta de novo.
Impacto nos utilizadores	As reservas falham temporariamente.
Perda de dados	Nenhuma. Sem duplicados, porque o pedido repetido leva a mesma chave de idempotência.
Como é feita a recuperação	A API volta a ligar-se quando a rede regressa.
Como se repõe a redundância	Não se aplica, porque nenhum componente se perdeu.

Como vamos testar

Cada cenário será demonstrado com utilizadores simulados a fazer reservas enquanto se desliga o componente (por exemplo com docker stop ou cortando a rede do contentor). Em cada teste registamos o tempo até o serviço voltar ao normal, o número de pedidos falhados e se houve reservas duplicadas ou perdidas (esperado: nenhuma). Testamos também muitos pedidos em simultâneo (por exemplo 50) para a mesma sala e confirmamos que só um é aceite.

Limitações

- O Nginx é único. Se falhar, o sistema pára. É uma limitação conhecida e não está coberta nesta versão.
- A promoção da réplica a primário é manual. Os tempos de recuperação serão medidos nos testes.
- A demonstração corre numa só máquina, por isso só simulamos falhas de contentores e de rede, não de máquinas inteiras.

Exemplo: falha do primário da base de dados

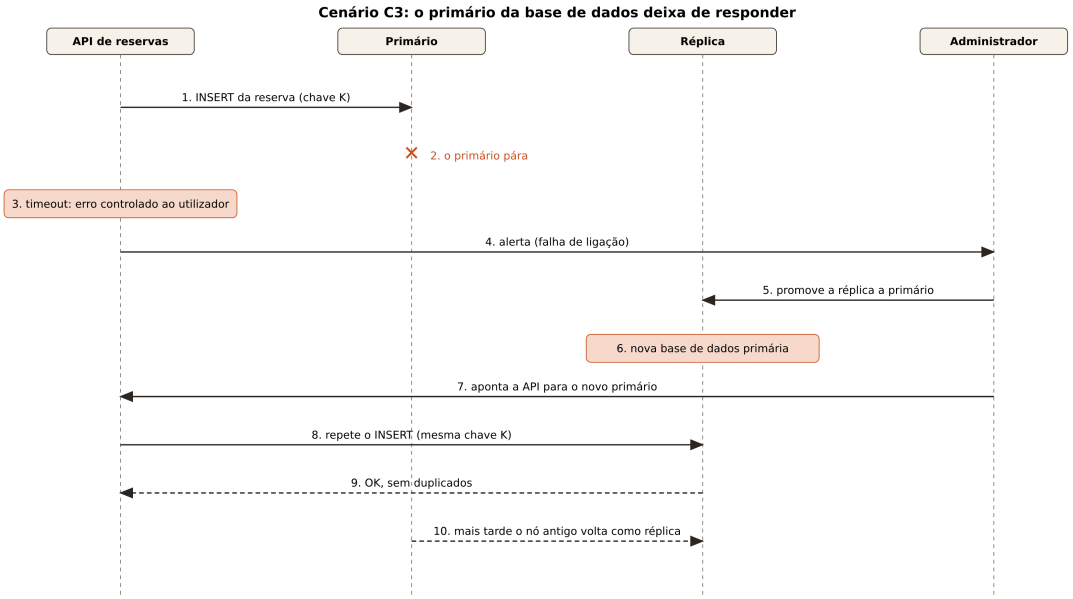


Figura 2. Cenário C3, passo a passo